

BCC1313 ALGORITHMS AND COMPLEXITY

Searching Techniques

INTRODUCTION

- **Searching algorithms** are methods used to locate specific elements or data within a collection, such as an array, list, or tree.

- Efficient searching is critical in applications like databases, file systems, and web search engines, as it optimizes data retrieval and processing.

APPLICATIONS OF SEARCHING ALGORITMS

- **Databases:** Searching algorithms speed up data retrieval from databases.
- **File Systems:** File search, data indexing, and retrieval.
- **Internet Search Engines:** Algorithms optimize how data is retrieved for search queries.
- **Pathfinding:** DFS and BFS are used in GPS systems, games, and AI for finding paths and connections.

SEARCHING ALGORITMS

- Searching algorithms play a pivotal role in data management systems, allowing for quick and efficient data retrieval, which is critical for performance in applications ranging from databases to search engines.
- Their efficiency directly impacts the speed at which users can access information and the overall responsiveness of applications.
- By employing appropriate searching algorithms, developers can enhance the performance of applications, especially when dealing with large datasets or real-time data processing.

LINEAR SEARCH

- **Linear search** is the simplest form of searching, where each element in a dataset is checked sequentially until the desired value is found or all elements have been checked.
- This approach does not require any specific order of the data. However, its time complexity is O(n), meaning that in the worst case, every element must be examined. This makes linear search less efficient for large datasets
- Example: Searching for a name in an unsorted list of names.

LINEAR SEARCH

Algorithm Steps:

1. Start from the first element of the list.
2. Compare the current element with the target value.
3. If a match is found, return the index of the current element.
4. If not, move to the next element and repeat the comparison.
5. Continue this process until the target is found or the end of the list is reached.

LINEAR SEARCH

- **Time Complexity:**
  - **Best Case: O(1)** - The target is found at the first position.
  - **Worst Case: O(n)** - The target is not present or is located at the last position, requiring a check of all n elements.
  - **Average Case: O(n)** - On average, the algorithm checks half of the elements.
- The performance of this search method decreases as the size of the dataset increases.

LINEAR SEARCH

When to Use Linear Search?

- Linear search is particularly useful in the following scenarios:
  - **Small Datasets**: When dealing with small arrays or lists
  - **Unsorted Data:** Linear search does not require data to be sorted, making it suitable for quick checks in unsorted collections.
  - **Simple Implementations**: For quick implementations in scripts or smaller projects - easier to code and understand.
  - **Data Streams**: When dealing with data streams or real-time data where the entire dataset cannot be held in memory, linear search can be applied iteratively.

LINEAR SEARCH

```
int linearSearch(int arr[], int size, int target) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == target) {
            return i;  // Return the index of the target
        }
    }
    return -1;  // Return -1 if the target is not found
}
```

```
int main() {
    int numbers[] = {4, 2, 5, 1, 3};
    int target = 1;
    int size = sizeof(numbers) / sizeof(numbers[0]);
    int result = linearSearch(numbers, size, target);
    cout << "Target found at index: " << result << endl;
    return 0;
}
```

NON-LINEAR SEARCH

- Non-linear search techniques refer to algorithms that utilize more complex data structures to optimize searching.
- These include methods such as **binary search**, which requires sorted data and significantly reduces the search space by repeatedly dividing the dataset in half. The time complexity of binary search is O(log n), making it much faster than linear search for larger, sorted datasets.
- Example: Finding a number in a sorted array of integers using binary search.

BINARY SEARCH

- Binary search significantly improves search efficiency by requiring that the dataset be sorted.
- By leveraging the order of the elements, binary search significantly reduces the number of comparisons required to find a value compared to linear search.
- The algorithm operates by repeatedly dividing the search interval in half.

BINARY SEARCH

- If the target value is less than the middle element of the array, the search continues in the lower half; if the target value is greater, the search continues in the upper half.
- This halving process continues until the target value is found or the search interval is empty

BINARY SEARCH

Algorithm Steps:

1. Start with two pointers: low (beginning of the array) and high (end of the array).
2. Calculate the middle index: mid = (low + high) / 2.
3. Compare the target value with the middle element:
4. If equal, return the mid index.
5. If the target is less than the middle element, adjust high to mid - 1 to search the left half.
6. If the target is greater, adjust low to mid + 1 to search the right half.
7. Repeat the process until the target is found or low exceeds high.
8. If the target is not found, return a designated value (e.g., -1).

BINARY SEARCH

Time Complexity:

- **Best Case: O(1)** - The target is found at the first check (middle of the array).
- **Worst Case: O(log n)** - The search space is halved with each iteration, making it much faster than linear search as the dataset grows.
- **Average Case: O(log n)** - On average, the number of comparisons remains logarithmic relative to the size of the dataset

BINARY SEARCH

- Binary search is ideal in scenarios such as:
  - **Sorted Datasets:** The dataset must be sorted prior to applying the algorithm. This can be an array, list, or any other ordered structure.
  - **Large Datasets:** As datasets increase in size, the efficiency of binary search becomes increasingly apparent, outperforming linear search by handling larger volumes of data in significantly fewer comparisons.
  - **Databases:** Binary search is often utilized in database indexing systems to speed up data retrieval processes.

BINARY SEARCH

Binary search is widely used in various applications, including:

- **Searching in Databases:** within their indexing systems to retrieve data efficiently.
- **Finding Elements in Sorted Arrays:** applied in scenarios where you need to quickly locate a value in a large, sorted list.
- **Library Management Systems:** To search for books or items by their IDs or any sorted property efficiently.
- **Algorithm Optimization:** for solving problems related to range queries and number finding.

BINARY SEARCH

- In comparing binary search to linear search, the key points to highlight include:
  - **Efficiency**: Binary search is significantly faster for large datasets, while linear search performs poorly as the size of the data increases.
  - **Dataset Requirements**: Binary search requires the dataset to be sorted, linear search can be applied to unsorted data.
  - **Use Cases**: Binary search is preferred in applications where frequent searches are required on static datasets, while linear search may be sufficient for simpler or smaller datasets.

SEARCHING IN DATA STRUCTURE

- Searching is a fundamental operation across various data structures, and each data structure provides unique methods and efficiencies for performing search operations.

SEARCHING IN DATA STRUCTURE

- **Searching in Arrays:**
  - Arrays offer a straightforward structure, where elements are stored sequentially and can be accessed by index.
  - **Linear Search:** In unsorted arrays, linear search is typically used. This involves sequentially checking each element.
  - **Binary Search in Sorted Arrays:** For sorted arrays, binary search can be used, dividing the search space in half with each step, achieving $O(\log n)$O(logn) efficiency.

SEARCHING IN DATA STRUCTURE

- **Searching in Arrays:**
  - **Advantages and Limitations:** Arrays provide fast access to elements by index, but their lack of dynamic memory allocation and need for shifting elements can make searches slow for unsorted or frequently modified data.

SEARCHING IN DATA STRUCTURE

- **Searching in Linked Lists:**
  - Linked lists store elements in nodes, where each node points to the next. Linked lists require linear search, as each node must be visited in sequence from the head to locate an element, resulting in $O(n)$O(n) search time.
  - **Advantages and Limitations:** Linked lists offer flexibility in memory allocation, but they are inefficient for search operations due to the lack of direct access to elements by index.

SEARCHING IN DATA STRUCTURE

- **Searching in Hash Tables:**
  - Hash tables use hash functions to map keys to indices, providing efficient access to data.
  - By computing the hash of a key, a hash table can quickly locate the bucket (or index) where the key-value pair is stored, achieving an average-case $O(1)$O(1) search time.

SEARCHING IN DATA STRUCTURE

- **Searching in Hash Tables:**
  - **Handling Collisions:** When multiple keys hash to the same index, collision resolution techniques like chaining or open addressing are used to maintain search efficiency.
  - **Advantages and Limitations:** Hash tables are highly efficient for searches but require careful management of hash functions and collision handling. They are unsuitable for ordered data retrieval and can consume more memory.

SEARCHING IN DATA STRUCTURE

- **Searching in Trees:**
  - Trees offer a hierarchical structure that enables efficient searching.
  - In a Binary Search Trees (BSTs), searching follows a left-right branching path, where values less than the current node go to the left child, and greater values go to the right, achieving $O(\log n)$O(logn) time complexity in balanced trees.

SEARCHING IN DATA STRUCTURE

- **Searching in Trees:**
  - **Traversal Searches:** Trees allow **depth-first and breadth-first traversal** methods, which are useful for exploring all nodes when needed (e.g., in decision-making or hierarchical analysis).
  - **Advantages and Limitations:** BSTs provide efficient searching and ordered data retrieval, but they can become unbalanced, degrading search efficiency to $O(n)$O(n).

SEARCHING IN TREES

- Searching within trees is an essential operation that allows efficient data retrieval and exploration, particularly in **binary trees** and **binary search trees (BSTs).**
- In a Binary Search Tree, searching is highly efficient due to the tree’s ordered nature. All nodes in the left subtree of a node are less than the node's key, while all nodes in the right subtree are greater.
- This property enables efficient binary search.

SEARCHING IN TREES

- Binary Search process begins at the root, the search algorithm compares the target value to the current node's value.
- If the target is less, the search moves to the left subtree; if greater, it moves to the right, repeating this process until the target is found or a null node is reached.

SEARCHING IN TREES

- **Depth-First Search (DFS) Traversal in Trees:**
  - DFS is a general tree traversal method that explores as far as possible along each branch before backtracking.
    1. **In-Order Traversal**: Visits nodes in ascending order in BSTs, making it useful for sorted data retrieval.
    2. **Pre-Order** and **Post-Order Traversals**: Often used in applications like expression parsing and building structured data representations.

SEARCHING IN TREES

- **Breadth-First Search (BFS) Traversal in Trees:**
  - BFS explores nodes level by level, moving horizontally across each level before descending.
  - BFS uses a queue to store nodes level by level. This method is efficient for finding the shortest path in unweighted trees and for exploring data structures where a level-wise view is beneficial.

SEARCHING IN TREES

- **Real-World Applications:**
  - **Databases and File Systems:** Trees are used in indexing, enabling efficient data retrieval and hierarchical storage.
  - **Routing Algorithms:** Tree structures are employed in network routing and pathfinding to optimize resource usage and connectivity.
  - **Game AI:** Game trees utilize searching algorithms to simulate moves and strategies in games, allowing AI to evaluate potential outcomes.

SEARCHING IN GRAPH

- Graph search algorithms like **Depth-First Search (DFS)** and **Breadth-First Search (BFS),** extend searching capabilities to more complex structures.
- DFS is an algorithm that explores as far as possible along each branch before backtracking, making it useful for traversing or searching tree and graph data structures.
- BFS explores all neighboring nodes at the present depth prior to moving on to nodes at the next depth level, making it ideal for finding the shortest path in unweighted graphs.

SEARCHING IN GRAPH

- Real-World Applications:
  - **Social Network Analysis**: Understanding how users are connected and identifying influencers within a network by analyzing paths between nodes.
  - **Routing Algorithms**: Optimizing routes for navigation systems or logistics companies by finding the shortest or most efficient paths through connected nodes.
  - **Network Connectivity**: Assessing the resilience of networks by determining connectivity and possible bottlenecks within a graph structure.

APPLICATION IN SEARCHING ALGORITHMS

- Searching algorithms are used in a variety of real-world applications, from databases to search engines.
- In **databases**, searching algorithms are fundamental for quickly locating and retrieving records from vast datasets. Techniques such as indexing are employed to improve search performance.
- **Search engines** utilize complex algorithms to provide users with relevant results. These algorithms are designed to search vast amounts of data stored in indexed formats.

APPLICATION IN SEARCHING ALGORITHMS

- **Query Optimization:** Searching algorithms are also vital for optimizing queries in relational databases. By analyzing query patterns and data access methods, these algorithms help streamline the retrieval process.
- In **data analytics**, searching algorithms enable the extraction of insights from large datasets by facilitating efficient data exploration and retrieval.

OPTIMIZING SEARCH ALGORITHMS

- As the volume of data grows and applications demand faster response times, optimizing search techniques becomes crucial.
- Other concepts such as caching, indexing, and data partitioning techniques, which collectively contribute to improving search efficiency.

OPTIMIZING SEARCH ALGORITHMS

1. **Caching:**
   - Caching involves storing frequently accessed data in a temporary storage location to reduce access times. By keeping a copy of data that is likely to be reused, search operations can be significantly accelerated.
   - **Advantages:** Reduces latency and improves the speed of data retrieval for repeated searches.
   - **Limitations:** Cache management can be complex, requiring strategies and may consume additional memory resources.

OPTIMIZING SEARCH ALGORITHMS

2. Indexing:

- Indexing creates a data structure (like a B-tree or hash table) that allows for quick lookups. It essentially provides a roadmap for the data, enabling faster searches without needing to scan the entire dataset.
- **Advantages:** Significantly reduces search time, particularly for large datasets.
- **Limitations:** Indexing can increase the time required for data insertion and updates, as the index must be maintained.

OPTIMIZING SEARCH ALGORITHMS

3. Data Partitioning:

- Data partitioning involves dividing a dataset into smaller, more manageable pieces or partitions based on various criteria, such as range (range partitioning), list (list partitioning), or hashing (hash partitioning), which can enhance search performance by limiting the amount of data that needs to be scanned.

OPTIMIZING SEARCH ALGORITHMS

3. Data Partitioning:

- **Advantages:** Makes searching more efficient by reducing the search space, especially in distributed systems where data is spread across multiple servers.
- **Limitations:** The overhead of managing partitions can be complex, and improper partitioning can lead to unbalanced loads, causing performance bottlenecks.

OPTIMIZING SEARCH ALGORITHMS

4. Dynamic Search Methods:

- Dynamic search methods adapt to changes in data and search patterns, which is essential for maintaining performance in environments where data is frequently updated.
- **Advantages:** Improves search efficiency by ensuring that the data structure remains optimized for current access patterns.
- **Limitations:** The complexity of maintaining such dynamic structures requires additional computational resources.

SUMMARY

- Choosing the right searching algorithm depends on factors like data structure, data order (sorted or unsorted), and specific application requirements.
- Key algorithms include **linear search**, suitable for sequential search in unsorted arrays and linked lists, and **binary search**, an optimized technique for sorted datasets.

SUMMARY

- Advanced data structures like **hash tables** provide nearly constant-time search through hash functions, while **trees**, particularly binary search trees, allow efficient, ordered data retrieval using hierarchical branching.
- Each data structure presents unique strengths and trade-offs.
- For example, **arrays** enable index-based access but are less flexible for dynamic data, while **linked lists** are ideal for frequent insertions but less efficient for search.

SUMMARY

- **Hash tables** excel at rapid lookups for large datasets, though they require careful collision management.
- **Trees**, particularly balanced trees, provide a structured approach to maintain search efficiency even as data grows.
- This knowledge serves as a foundation for advanced topics, such as search optimization techniques and applications in real-world scenarios like database indexing, network routing, and AI pathfinding.